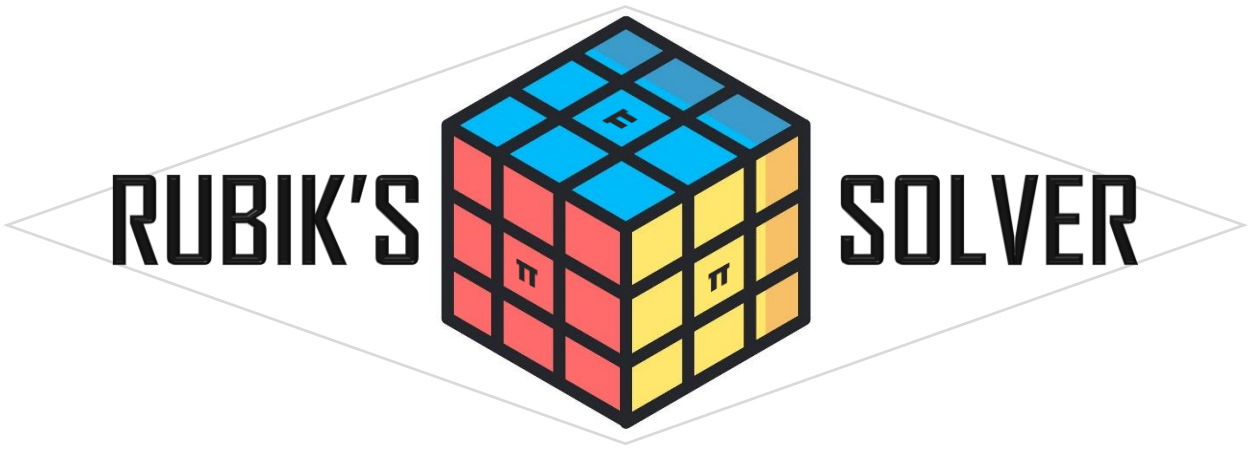
SYSC3010

Computer Systems Development Project

π^3



Group L3-G6

Luke Grundy, 101268449
Basil Thotapilly, 101313150
Eric McFetridge, 101310942
Saim Hashmi, 101241041

TA: Gabriel Seyoum

1/30/2026

Table of Contents

1 Introduction

This document was produced in response to a requirement of SYSC 3010 where students are tasked to develop and implement a project as a group. Our group (L3-G6) is proposing π^3, a modular Rubik's cube solving platform that allows the user to explore the functionality of Rubik's cubes while gaining knowledge in algorithms and robotics. This proposal will provide general information on the π^3 project including its motivations, objectives and goals. The document will also provide insight into the system's design, its components and the use cases of the project. In conclusion the proposal will discuss the development plan for the project, describing its work plan, technical requirements and any additional hardware that will be required.

1.1 Background

Modern day off-the-shelf Rubik's cube solving machines can solve cubes quickly while providing a clean easy to use software interface. However, these systems are largely contained and function as a unified service, hindering users from exploring the functionality of the machine. If a person wants to have their machine solve the cube faster or show the process the machine uses to solve the cube, they are unable to do so with their hardware. A path these people can take is making a custom cube solver. Custom cube solvers can be classified into two separate groups, instructional designs (step by step guides) and proprietary designs (custom personal projects). Instructional designs allow for more exploration into the mechanisms of its system while providing an easy process to complete the project. These kinds of designs allow for more customizability compared to an off-the-shelf product but are built for only one kind of form factor and don't allow for large design modifications. Proprietary designs allow for the most freedom in how the solver behaves but requires a large amount of development time.

1.2 Motivation

With our group's passion for Rubik's Cubes, along with the interesting design challenges that come with the project, π^3 will be a learning tool for robotics and algorithms as well as a stepping stone project for advanced users to design their own cube solver. The Rubik's Cube is an amazing tool for teaching children or new students about the concept of algorithms. You take the cube scramble, apply the selected algorithm, and receive a new output from that algorithm that in our case is a solved cube. The Rubik's cube is also an engaging puzzle that teaches problem solving, pattern recognition, and enhances long term cognitive ability. [5]

1.3 Project Objective

The objective of this project is to develop a system that can scan a Rubik's cube using a digital camera module and convert that 3x3x3 layout to a data structure. Using that data structure, a second module produces a valid solution for the current cube state. This solution is converted to a series of ninety-degree rotations. Another module, the motor control system, takes these rotations and executes them in order. This allows the system to spin any 5/6 faces on the cube. There is no motor located on the top face to facilitate sliding the cube in and out of the rotation system. The objective is not just to solve the cube, but rather to teach the user the math and science concepts that explain how it functions. By allowing the user to adjust parameters like which solving algorithm to use, and how fast to spin the cube, the system no longer has a single purpose but can provide an interactive learning platform to exercise the users' brain. Users can compare metrics on the effect of different algorithms on average speed to solve and get human readable instructions to solve the cube themselves after scanning it.

1.4 Specific Goals

Functional Requirements:

Cube Scanning: The system shall recognize the state of the input Rubik's cube using 2 cameras and send it to the server as a data structure that can be stored in the database.

Solving algorithm: The system shall retrieve the cube state from the DB and generate a solution using the selected algorithm.

Solving Output: The system shall use the CFOP method for solving and send the solution to the server in the form: Move # | Face # | Direction.

Cube permutations: The system shall use 5 motors to rotate the cube based on commands received from server module. The system should solve the cube in less than 15 seconds.

System GUI: The system shall display a 3D cube model and solving steps on a web dashboard accessed by a phone or computer.

Notification system: The system shall send a notification to a connected device in the event of a fatal error (disconnected motor/camera, unresponsive pi)

Solving Pipeline: The system shall process the data in the following order: Scan, Solve, Rotate, Verify.

Connectivity: The system shall be able to send data between raspberry PIs using TCP.

Nonfunctional Requirements:

Reliability: The system shall detect an unresponsive Pi within 5 seconds using heartbeats and transition the job into an error state without continuing execution.

Safety: If a fatal error occurs during execution, the motor subsystem shall stop within 1 second of receiving a stop signal, and the system shall require a manual reset before restarting execution.

Usability: The GUI shall be understandable by a first-time user and provide clear labels, status indicators, and recovery steps when errors occur.

Maintainability: Each subsystem shall be independently testable using well-defined inputs/outputs (database tables and message formats) so that components can be debugged in isolation.

2 System Design

2.1 System Overview Diagram

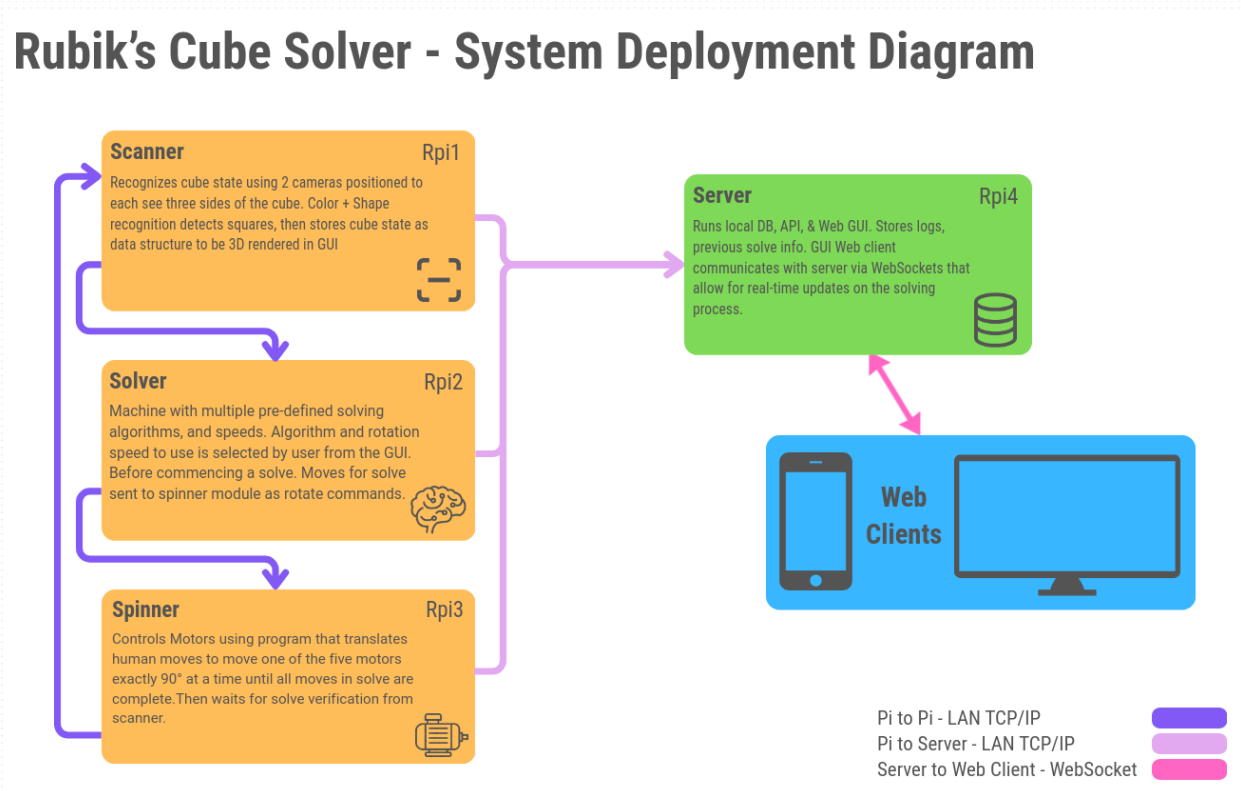


Figure 1: Deployment diagram for the Rubik's Cube Solver.

Figure 1 presents the high-level deployment of Rubik's Cube Solver. The system is intentionally distributed across multiple Raspberry Pis to keep each subsystem modular and independently testable. The Cube Scanning Pi is responsible for image capture and cube-state extraction; the Solver Algorithm Pi generates a valid solution sequence, and the Motor Control Pi performs time-critical actuator control. A dedicated Database & GUI Pi hosts a centralized database for shared state, error logging, and coordination, as well as the web-based interface used by the user.

This structure reduces coupling between subsystems and improves debugging. Each Pi publishes its outputs to the database (e.g., cube state; solution moves, motor progress) and consumes only the data needed for its next step. The Database & GUI Pi acts as the "single source of truth" for the pipeline state (Scan → Solve → Execute), allowing the system to recover more cleanly from partial failures and making it easier to verify integration milestones.

2.1.1 Communication Protocols

All Raspberry Pis communicate over the local Wi-Fi network using TCP/IP for guaranteed local data delivery with minimal overhead. Each pi simply has a dedicated TCP socket where info is sent and received by the server. The system does not need to be exposed to the internet to function, unless the user desires to port forward their web dashboard for outside access. System coordination is implemented through a centralized database hosted on the Database & GUI Pi. Each subsystem reads and writes structured records to the database to exchange information and to signal progress.

Inter-node communication (Pi ↔ Pi via Database & GUI Pi)

- Cube Scanning Pi → Database: Writes the detected cube state for the active job, including a timestamp and a confidence/validation flag.
- Solver Algorithm Pi ↔ Database: Reads the latest valid cube state, writes back the computed solution (move sequence) plus metrics such as compute time and move count.
- Motor Control Pi ↔ Database: Reads the solution moves (or a command queue) and writes back live execution status (current move index, completion status, and any error codes).
- All Pis → Database: Write periodic heartbeat timestamps so the system can detect failures quickly.
- Database & GUI Pi → User GUI: The web server reads the current job state from the database and displays live status. GUI actions (Start/Stop/Reset/Rescan) are written as control signals for other subsystems to observe.

Hardware communication (Pi ↔ Devices)

- Cameras ↔ Cube Scanning Pi: Two cameras connect via USB (or CSI) and are accessed through the camera driver stack. Frames are processed locally to detect sticker colors.
- Motors ↔ Motor Control Pi: Motors are driven through motor driver modules controlled by the Pi's GPIO. The motor controller uses STEP/DIR (or equivalent) signals and enforces safe motion limits and stops behavior.

Message format and ownership to avoid conflicts, each subsystem "owns" the data it produces:

- Scanning Pi owns cube state records.
- Solver Pi owns solution records.
- Motor Pi owns execution status records.
- Database & GUI Pi owns the job state machine, logging, and notification triggers.

2.2 Component Details

2.2.1 Component 1 - Scanner Pi

Primary task: The Scanner Pi is responsible for scanning and interpreting the current state of the Rubik's cube. This subsystem consists of a Raspberry Pi connected to 2 cameras used to capture images of the cube's faces. Using computer vision techniques, the color of each sticker is identified, and a digital representation of the cube state is created. The state of the cube is then stored in the database and made available for the solving algorithm subsystem to use. The scanning subsystem also performs basic validation of the detected cube state and reports scanning status and errors.

2.2.2 Component 2 - Solver Algorithm Pi

Primary task: The solver algorithm Pi is responsible for finding a solution of a given cube. It receives the cubes state stored in the database pi and attempts to find the most efficient solution using an algorithm selected by the user. By default, the pi will use an algorithm called CFOP (Cross, F2L, OLL, PLL), which is the most common speed cubing algorithm in the competitive scene.

While this algorithm is not the most efficient solving method that a computer is capable of, it is the ideal algorithm to use in our group use case. There are two factors contributing to the use of CFOP: hardware design and educational benefits. With the 5 motors being used to apply the solution pattern to the Rubik's cube, one face is not accessible to rotate. The CFOP algorithm library gives enough alternate algorithms to keep one face stationary while achieving a competitively low number of turns needed for a solve. Using CFOP also allows

for a tutorial mode to be used to practice algorithms. As CFOP is a method that humans can memorize, having a tool to easily practice certain algorithms will allow for faster learning in cases less commonly encountered while solving.
While CFOP is already a quick way to solve a Rubik's cube, the use of a computer allows this method to be even more effective. Instead of finding only one solution for the input cube, the solver will find many possible solutions and compare the solve times to find the solution with the least moves. After the solution is found the string of moves is encoded and sent to the server Pi to be displayed and delivered to the motor control pi.

2.2.3 Component 3 - Motor Control Pi

Primary task: The Motor control Pi receives encoded instructions from the server Pi which has a translated copy of the cube solve pattern that corresponds to a series of motor movements. The motors have two operation modes; the first is full speed, where the motors rotate at their physical limits as fast as possible, while still ensuring move integrity and cube positioning. The second mode is running at a user-defined speed, such as 1 full rotation per second. This is easily done by artificially delaying the next rotate command sent to the motors using a timer to interrupt or a software sleep(). To control the motors, the Pi will communicate with a dedicated motor controller bridge IC. The bridge circuit will provide a control interface, and provide the external power needed for the higher voltage motors. This allows us to control multiple motor speeds and directions simultaneously by using the interface and the Pi's GPIO.

Core Responsibilities:

1. Ensure Rotation occurs within the time window selected by the user. Max speed to be determined in the testing phase.
2. Ensure rotation is exactly 90 degrees so cube does not jam during movement
3. Ensure rotation happens in correct order corresponding to the solution for the currently selected algorithm.
4. Ensure cube is in solved state after final rotation (additionally verified by scanner)

Interfaces:

1. TCP socket for receiving move list from server. (Input)
2. Motor bridge for hardware control. (Output)

Error handling:

Upon motor health check, a failure message is sent to server Pi with error saying motor subsystem can't be reached. System enters failure mode and prompts user on GUI to verify physical connections and power. Also pets heartbeat timer every few seconds to ensure GUI/DB system error gracefully on sudden loss of connection or missing heartbeat.

2.2.4 Component 4 - Database & GUI Pi

Primary task: The Database & GUI Pi hosts the centralized database used for inter-Pi coordination and provides a web-based GUI for controlling the pipeline and monitoring system status. It also implements system coordination and logging so that every solve attempt can be traced and debugged.

Core responsibilities:

1. Central database hosting:
- Store cube states, solver outputs, motor execution progress, and system logs in a single place accessible by all subsystems.
- Maintain a "job" record for each solve attempt to keep the pipeline state consistent.
2. System coordination (state machine):
- Manage job states: Idle → Scanning → Solving → Executing → Done/Error.
- Enforce ordering so Solve only starts after Scan is valid, and Execute only starts after Solve is complete.
- Detect missing heartbeats and automatically transition the job to Error if a subsystem becomes unresponsive.
3. Web-based GUI:
- Provide a browser-accessible interface for phone or computer.
- Allow user actions: Start Solve, Stop, Reset, Rescan.
- Display: current stage, progress, last cube state, move list preview, and error messages with recovery guidance.
4. Logging and notification:
- Store error and event logs (camera disconnect, motor stall, unresponsive Pi).
- Trigger notifications through the GUI (banner + browser notification) when a fatal error is recorded.

Interfaces (inputs/outputs): Inputs:

- cube_state updates from the Scanning Pi.
- solution_moves from the Solver Pi.
- motor progress and completion from the Motor Control Pi.
- heartbeat messages from all Pis.
- user control actions from the GUI.

Outputs:

- job state transitions written to the database.
- control flags (start/stop/reset/rescan) visible to other Pis.

- error notifications and visible status changes in the GUI.

Preliminary database schema (high-level) A) jobs

- job_id (primary key)
- created_ts
- state (Idle/Scanning/Solving/Executing/Done/Error)
- active_flag
- last_update_ts

B) cube_states

- job_id (foreign key)
- ts
- state_string (54 facelets or equivalent standardized representation)
- confidence
- is_valid

C) solutions

- job_id (foreign key)
- ts
- method (CFOP)
- moves (sequence in notation, e.g., “R U R’ U’ …”)
- move_count
- solve_time_ms

D) execution_status

- job_id (foreign key)
- ts
- current_move_index
- status (InProgress/Done/Error)
- error_code (nullable)

E) events (logging)

- event_id (primary key)
- ts
- pi_name
- severity (Info/Warning/Error/Fatal)
- message

F) heartbeats

- pi_name (primary key)
- ts_last_seen
- status (OK/Degraded/Offline)

GUI pages (minimum set)

1. Dashboard:
- Job state, subsystem online/offline, progress bar, last update time, last error.
2. Solve View:
- Shows last scanned cube state (text + optional visualization) and the computed move sequence.
3. Controls:
- Start / Stop / Reset / Rescan buttons with confirmation prompts for Stop/Reset.
4. Logs:
- Recent events, filterable by Pi and severity.

Failure handling behavior: If any Pi misses heartbeats for >5 seconds, the system records a Fatal event, transitions the active job to Error, and disables execution until the user resets. If a motor/camera disconnect is reported, the system immediately triggers a GUI notification and records the event for debugging.

2.3 Use Cases

Use Case Name	Normal Solve Flow
Use Case #	**1**
Description: This use case describes the primary success path where the user starts a solve, the cube is scanned, a solution is computed, and the motor subsystem	
Steps: 1) User presses “Start Solve” in the web GUI. 2) Database & GUI Pi creates a new job record and sets state = Scanning. 3) Cube Scanning Pi performs scanning and writes cube state (valid) to the database. 4) Database & GUI Pi transitions job to Solving. 5) Solver Algorithm Pi reads cube state, computes CFOP solution, and writes solution moves to the database. 6) Database & GUI Pi transitions job to Executing. 7) Motor Control Pi reads the solution moves, executes them, and updates execution status after each move. 8) Motor Control Pi writes completion confirmation; Database & GUI Pi sets state = Done.	

9) GUI displays success summary (scan time, solve time, execution time, move count).

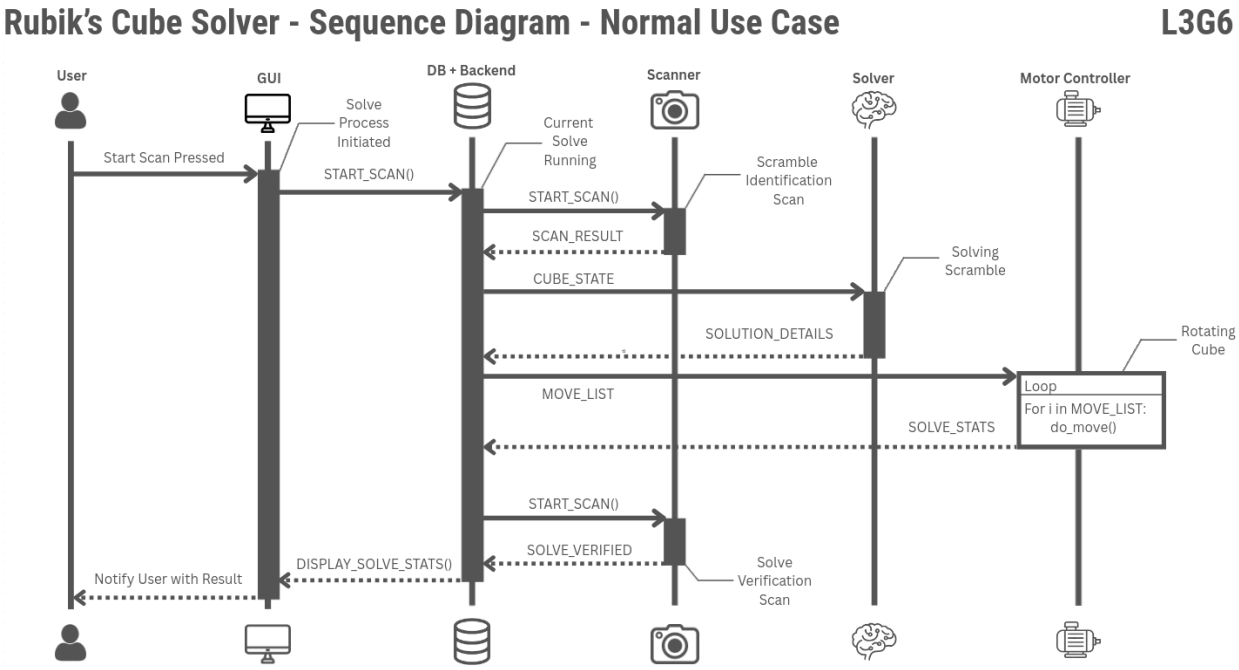


Figure 2: Use case 1 sequence diagram

Use Case Name	Fatal Error and Notification
Use Case #	**2**
Description: This use case describes how the system responds to a fatal fault such as a disconnected motor/camera or an unresponsive Pi.	
Steps: 1) A subsystem detects a fatal condition (e.g., camera disconnect, motor stall, or heartbeat timeout). 2) The subsystem writes a Fatal event to the events table (or updates execution status with Error). 3) Database & GUI Pi transitions the active job to Error and writes a stop control signal (if executing). 4) Motor Control Pi stops motion immediately (if applicable) and confirms stop. 5) GUI displays an error banner and triggers a browser notification with recovery steps (e.g., "Check camera cable and rescan."). 6) User presses Reset to clear the job and return the system to Idle.	

Figure 3: Use case 2 sequence diagram

Use Case Name	Rescan / Retry
Use Case #	**3**
Description: This use case describes how a user retries scanning when a scan is invalid or confidence is low.	
Steps: 1) GUI indicates scan invalid/low confidence and suggests Rescan. 2) User presses Rescan. 3) Database & GUI Pi clears the prior cube state for the active job (or opens a new job) and sets state = Scanning. 4) Cube Scanning Pi repeats scan and writes the new cube state. 5) Pipeline continues to Solving and then Executing if the new scan is valid.	

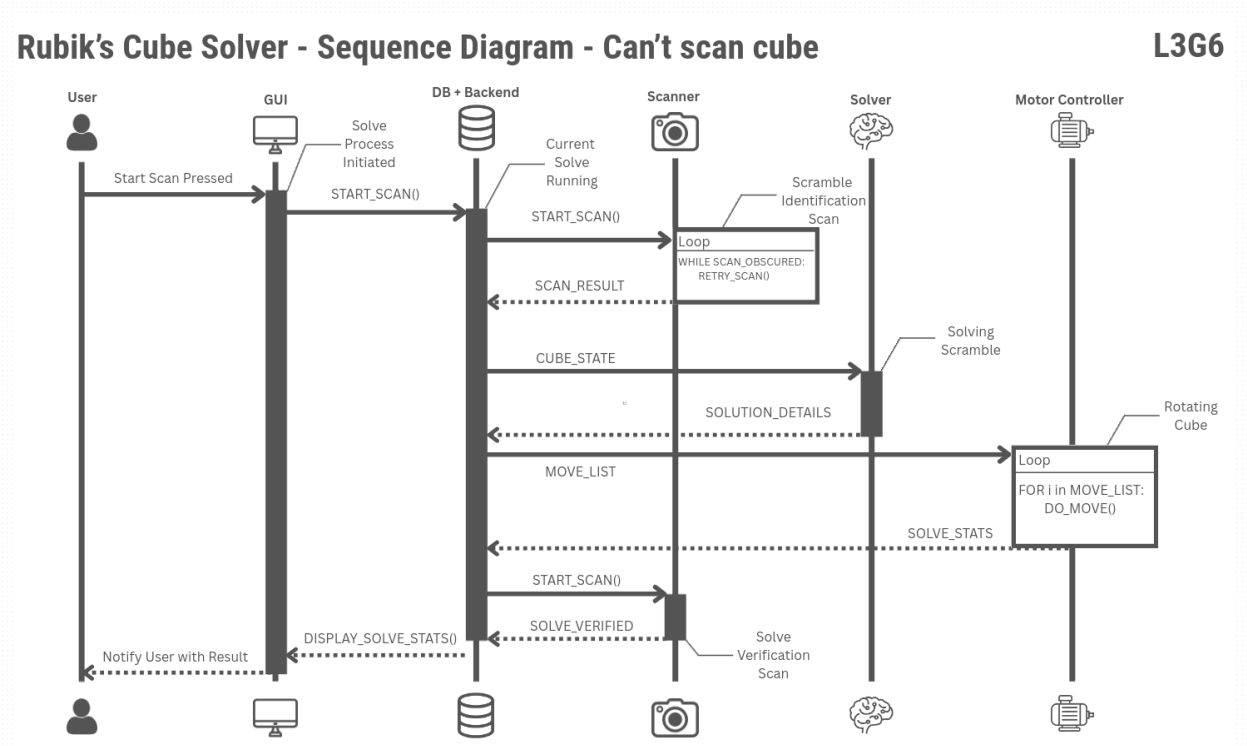


Figure 4: Use case 3 sequence diagram

3 Work Plan

3.1 The Project Team

Eric McFetridge – Background in UI/UX design, ipv4 networking, Linux systems, and embedded systems on the RPi/Arduino platforms.

Basil Thotapilly - Background in embedded systems and hardware interfacing.

Luke Grundy – Background in robotic systems using Arduino, Rubik's Cube solving, graphics design.

Saim Hasmi – Background in full stack Web applications and SQL Databases

3.1.1 Roles and Tasks

Team Member	Primary Responsibility	Secondary Responsibility
Luke Grundy	Solver Algorithm Pi	Motor command formatting and error handling
Basil Thotapilly	Cube Scanning Pi	Database integration and scan verification
Eric McFetridge	Motor Control Pi	Integration testing with solver commands
Saim Hashmi	Database and GUI Pi	System coordination and logging

3.1.2 Teamwork Strategy

Our team will work using a modular development strategy which is aligned with the distributed architecture of the system. Each team member will be primarily responsible for one raspberry Pi subsystem (motor control, cube scanning, solving algorithm, or database/GUI) while also assisting with other areas as needed.

Microsoft Teams will be used as the primary communication platform for weekly meetings and any discussions relating to the project. Project tasks, milestones, and due dates will be tracked using Notion allowing the team to effectively assign responsibilities, monitor progress and manage dependencies between subsystems.

3.1.3 What we will need to learn

To complete this project the team will need to learn and apply several new technologies:

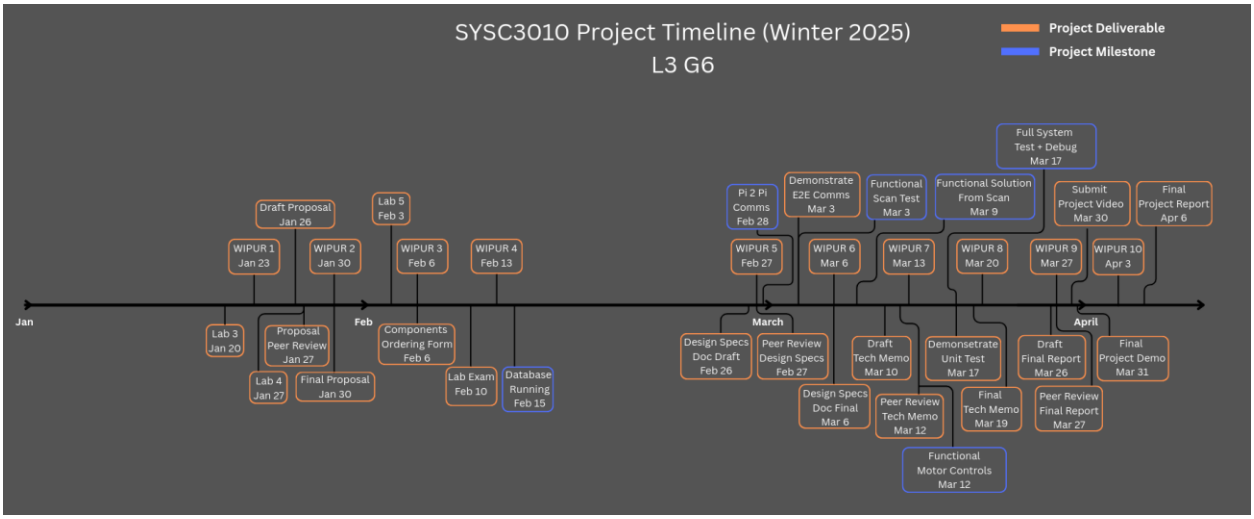
- Computer Vision for Cube State Detection: We will learn how to use camera input to detect and distinguish the Rubik's cube sticker colours effectively.
- Rubik's Cube Solving Algorithms: We will learn how to represent the cube's states in a program and develop a solving algorithm with inspiration from existing algorithms

- Motor Control: We will learn how to control motors through Raspberry Pi GPIO which will include timing, direction control and safety considerations.
- Web-based GUI and 3D Visualization: We will learn how to create a web-based GUI that displays a 3D model and allows user interaction.

3.2 Project Milestones

Milestone Number	Milestone Name	Description	Deadline
1	Database running	The database Pi is running a local database with the required tables	February 15th
2	Inter Pi Communication Established	All Raspberry Pis can successfully read from and write to the shared database.	February 28th
3	Cube Scanning Functional	The scanning Pi can capture images of the cube, detect sticker colours and generate a valid cube state stored in the database.	March 3rd
4	Solver Algorithm Integrated	Given a valid cube state from the database the solver Pi can generate a complete sequence of moves that solve the cube.	March 9th
5	Motor Control Execution	The motor control Pi can correctly execute individual cube rotations based on received commands and send confirmation of completion to the database.	March 12th
6	System Verification and Debugging	The fully integrated system can be tested to identify and correct faults across all subsystems	March 17th

3.3 Schedule of Activities



4 Project Requirements Checklist

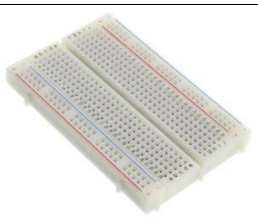
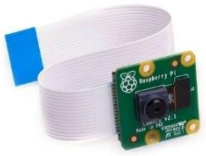
Requirement	Satisfied? (Y/N)	How Requirement Is Satisfied
Is there a computer (RPi, Arduino, Launchpad, etc.) per student in the group?	Y	4 Pi's & 4 members, scanner Pi, solver Pi, motor control Pi, and server/DB Pi
Is at least one RPi in headless mode?	Y	All Pi's will be headless in implementation
Is there at least one hardware device per student in the group?	Y	A minimum of one components is needed. Ours are the motors x5, and the cameras x2. 1 input 1 output.
Is there an actuator? (i.e., not every hardware device is a sensor)	Y	The motors are the actuators in our system
Is there a feedback loop? Interaction between input/output devices	Y	Feedback loop created by scanning the cube after spinning to verify if solve was successful.
Is there a database?	Y	Yes, for solving history and logs.

Does the computer hosting the database have other responsibilities?	Y	Yes, the computer also hosts the web backend/API for the GUI.
Does the database contain at least two tables (or types of information)?	Y	Yes, one table for solve history, another table for system logs.
Is there a periodic timing loop?	Y	Yes, machine enters periodic solve state where moves are completed every x ms "*for i in num_moves"*
Is there some processing or analysis of the IoT data read?	Y	Yes, scanned cube state is processed into string of move directions by solving algorithms.
Are notifications sent (SMS, email, push notifications)?	Y	Yes, push notifications sent on error events containing error messages.
Is there at least one bidirectional GUI running on a computer?	Y	Yes, GUI displays 3D render of scanned cube state and allows for adjusting solve algorithms & speed.

5 Additional Hardware Required

The Implementation of the project will require additional hardware included in the table below.

Component Name	Picture	Quantity
Stepper motor		5
Stepper motor driver		

Breadboard		2
Camera		2
Wires		25-100
3d Print time + filament		

6 References

{Saim}
[1] Speedsolving.com Wiki, “CFOP method,” Accessed: 2026-01-26. [Online]. Available: https://www.speedsolving.com/wiki/index.php/CFOP_method
[2] OpenCV, “OpenCV Documentation,” Accessed: 2026-01-26. [Online]. Available: https://docs.opencv.org/4.x/
[3] Raspberry Pi Ltd., “Raspberry Pi Documentation,” Accessed: 2026-01-26. [Online]. Available: https://www.raspberrypi.com/documentation/
[4] Raspberry Pi Ltd., “Raspberry Pi OS Documentation (GPIO from Python),” Accessed: 2026-01-26. [Online]. Available: https://www.raspberrypi.com/documentation/computers/os.html
[5]